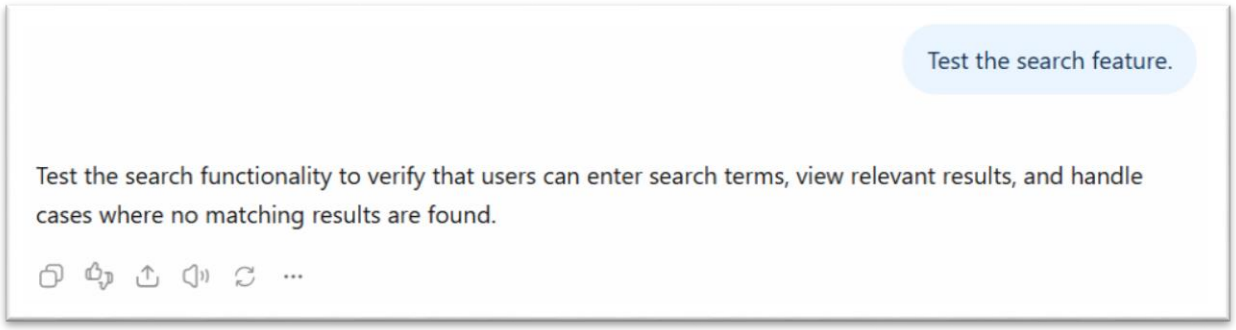
Scenarios-Prompthaton

- **Scenario 1 — Search Functionality**

Weak Prompt: "Test the search feature."



ICEPOT Prompt:

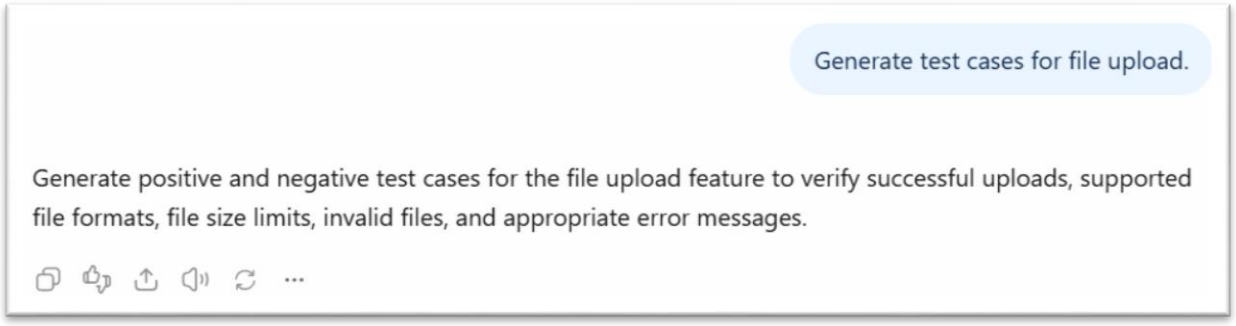
I(Instruction)	[STRICT] [MANDATORY] Generate positive & negative test cases for the pharmacy application's medicine search page. [MUST NOT] Generate duplicate test cases.
C(Context)	[IMPORTANT] The application allows users to search for medicines by name.
E(Example)	One test case should verify that searching for a valid medicine displays the matching results in the search results grid.
P(Persona)	Act as an experienced software test professional.
O(Output Format)	[EXACT] Return the test cases in a table with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

Positive and Negative Test Cases for the Pharmacy Search Page

Test Case ID	Steps	Test Data	Expected Result
TC_001 – Positive	1. Open the pharmacy application. 2. Enter a valid medicine name in the search field. 3. Click the Search button.	Paracetamol	Matching medicine details should be displayed in the search results grid.
TC_002 – Positive	1. Enter a complete medicine name. 2. Click Search.	Amoxicillin	The correct medicine should appear in the search results grid.
TC_003 – Positive	1. Enter a partial medicine name. 2. Click Search.	Parac	Matching medicines containing the entered text should be displayed, if partial search is supported.
TC_004 – Positive	1. Enter a medicine name using lowercase letters. 2. Click Search.	paracetamol	Matching results should be displayed regardless of letter case, if search is case-insensitive.
TC_005 – Positive	1. Enter a medicine name with leading and trailing spaces. 2. Click Search.	`Paracetamol`	The application should handle extra spaces and display matching results.
TC_006 – Positive	1. Enter a valid medicine name. 2. Press Enter.	Ibuprofen	The search should execute and display the matching results.
TC_007 – Negative	1. Enter a medicine name that does not exist. 2. Click Search.	XYZMedicine123	A clear "No results found" message should be displayed.
TC_008 – Negative	1. Leave the search field empty. 2. Click Search.	Blank	The application should display a validation message or handle the empty search according to the requirements.
TC_009 – Negative	1. Enter only spaces in the search field. 2. Click Search.		The application should treat the input as empty and display appropriate validation or guidance.
TC_010 – Negative	1. Enter special characters in the search field. 2. Click Search.	`@#$%^&*`	The application should handle the input safely without errors or unexpected results
TC_011 – Negative	1. Enter numeric values only. 2. Click Search.	`123456789`	The application should display matching results if supported, or a suitable no-results message.
TC_012 – Negative	1. Enter a medicine name with a spelling mistake. 2. Click Search.	Paracetmol	The application should display a no-results message or relevant suggestions, according to the requirements.
TC_013 – Negative	1. Enter an extremely long search string. 2. Click Search.	A string of 500 characters	The application should handle the input without crashing or displaying a system error.
TC_014 – Negative	1. Enter a search term containing SQL-like characters. 2. Click Search.	`' OR '1'='1`	The application should process the input safely and must not expose unauthorized data or cause a system error.
TC_015 – Negative	1. Enter a valid medicine name. 2. Disconnect the network before searching. 3. Click Search.	Paracetamol	The application should display an appropriate network or connection error message.

- **Scenario 2 — File Upload**

Weak Prompt: "Generate test cases for file upload."



ICEPOT Prompt:

I(Instruction)	[STRICT] [MANDATORY] [NON-DUPLICATE] [POSITIVE] [NEGATIVE] [EDGE] Generate positive, negative, and edge test cases for file upload functionality.
C(Context)	[IMPORTANT] The banking application enables users to upload KYC documents during account opening. Supported file formats are PDF, JPG, JPEG, and PNG, with a maximum file size of 5 MB per file. The system must validate both file extensions and actual content, reject malicious or corrupted files, and enforce appropriate access controls.
E(Example)	One test case should verify that file upload is successfully done with green tick mark.
P(Persona)	Act as a Senior QA Engineer with 10 years of experience.
O(Output Format)	[EXACT] Return the test cases in a table with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

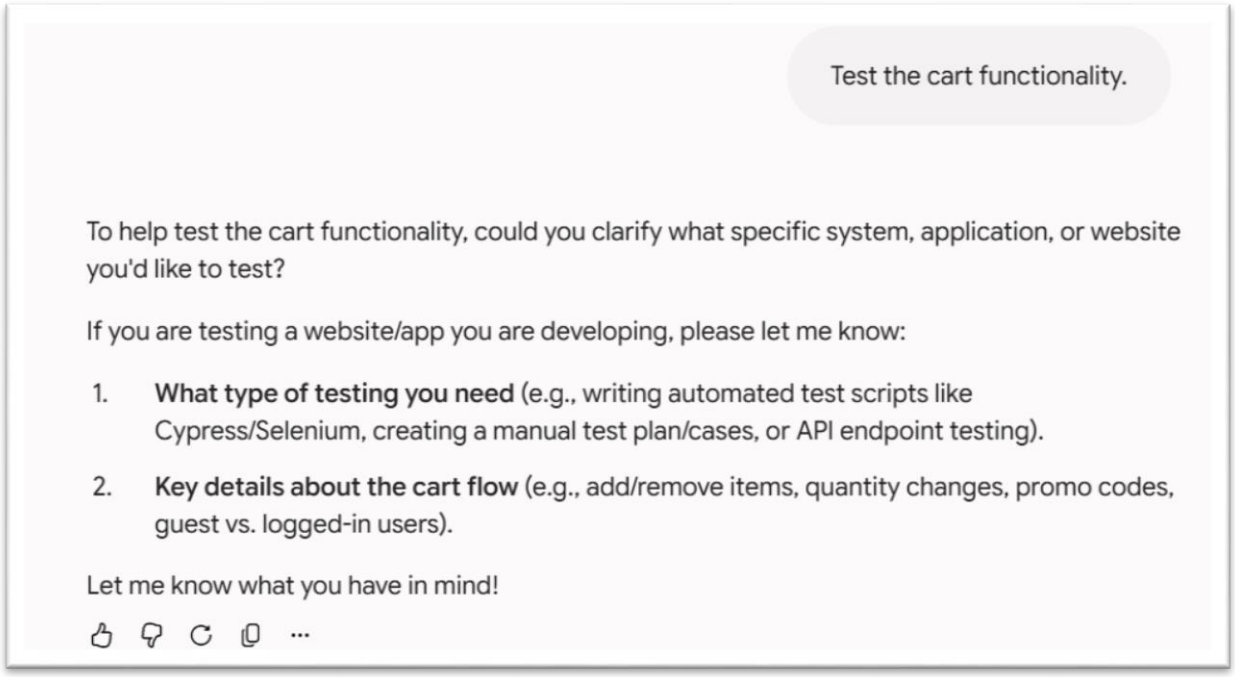
File Upload Test Cases — Banking KYC Verification

Test Case ID	Steps	Test Data	Expected Result
TC_001 [POSITIVE]	1. Navigate to the KYC upload page. 2. Select a valid PDF file. 3. Click Upload.	`identity.pdf`, 2 MB	File uploads successfully, and a green tick mark is displayed.
TC_002 [POSITIVE]	1. Open the KYC upload page. 2. Select a JPG file. 3. Upload the file.	`photo.jpg`, 1 MB	File uploads successfully, and the upload status is displayed.
TC_003 [POSITIVE]	1. Select a JPEG file. 2. Upload the file.	`address.jpeg`, 3 MB	File uploads successfully.
TC_004 [POSITIVE]	1. Select a PNG file. 2. Upload the file.	`document.png`, 2 MB	File uploads successfully.
TC_005 [POSITIVE]	1. Select a valid file at the maximum permitted size. 2. Upload the file.	`identity.pdf`, exactly 5 MB	File uploads successfully, and a green tick mark appears.
TC_006 [POSITIVE]	1. Log in as an authorized user. 2. Upload a valid KYC document.	`identity.pdf`, 2 MB	The authorized user can upload the document successfully.
TC_007 [NEGATIVE]	1. Select an unsupported file. 2. Attempt to upload it.	`document.exe`	Upload is rejected with an appropriate file-format error message.
TC_008 [NEGATIVE]	1. Select a PDF file exceeding the size limit. 2. Upload it.	`ider .pdf`, 5.1 MB	Upload is rejected with a file-size validation message.
TC_009 [NEGATIVE]	1. Open the upload page. 2. Click Upload without selecting a file.	No file selected	The system prevents submission and displays a required-file message.
TC_010 [NEGATIVE]	1. Select a corrupted PDF file. 2. Attempt to upload it.	`corrupted.pdf`, 1 MB	The system rejects the corrupted file and displays an appropriate error message.
TC_011 [NEGATIVE]	1. Select a malicious file renamed with a supported extension. 2. Attempt to upload it.	Malicious content named `identity.pdf`	The system detects invalid or malicious content and rejects the file.
TC_012 [NEGATIVE]	1. Log in as an unauthorized user. 2. Attempt to access the KYC upload function.	Unauthorized user account	Access is denied, and the user cannot upload KYC documents.
TC_013 [NEGATIVE]	1. Select a file with a supported extension but invalid content. 2. Upload it.	Text file renamed to `identity.pdf`	The system validates the actual file content and rejects the file.
TC_014 [NEGATIVE]	1. Select a zero-byte file. 2. Attempt to upload it.	`empty.pdf`, 0 bytes	The system rejects the empty document if empty files are prohibited.
TC_015 [EDGE]	1. Select a valid file just below the maximum size. 2. Upload it.	`identity.pdf`, 4.99 MB	File uploads successfully.
TC_016 [EDGE]	1. Select a valid file one byte above the maximum size. 2. Upload it.	`identity.pdf`, 5 MB + 1 byte	Upload is rejected with a file-size validation message.

TC_017 [EDGE]	1. Select a file with a long filename. 2. Upload it.	`customer_identity_verification_document_2026.pdf`	The system handles the filename according to its configured filename restrictions.
TC_018 [EDGE]	1. Select a valid file. 2. Click Upload multiple times rapidly.	`identity.pdf`, 2 MB	The system prevents unintended duplicate uploads or processes requests according to the defined requirements.
TC_019 [EDGE]	1. Select a valid file. 2. Start uploading. 3. Disconnect the network during upload.	`identity.pdf`, 2 MB	The system displays an appropriate failure or retry message and does not falsely indicate successful upload.
TC_020 [EDGE]	1. Upload a valid KYC document. 2. Attempt to access the uploaded document using another user's account.	Document belonging to User A; access attempted by User B	Access is denied unless explicitly authorized, protecting the document from unauthorized access.

- **Scenario 3— Shopping Cart**

Weak Prompt: "Test the cart functionality."



ICEPOT Prompt:

I(Instruction)	[STRICT] [MANDATORY] [NON-DUPLICATE] [POSITIVE] Generate positive smoke test cases for Crackers shopping cart functionality.
C(Context)	[IMPORTANT] The Crackers shopping application allows users to browse available crackers, add products to the shopping cart, view selected items, update quantities, remove products, and verify the cart total. The cart should display accurate product details, prices, and quantities and allow users to proceed to checkout when the required conditions are met.
E(Example)	One test case should verify that a user can successfully add a cracker product to the shopping cart and that the cart displays the correct product name, price, quantity, and total amount.
P(Persona)	Act as a QA Engineer with strong domain knowledge and a clear understanding of business requirements and application functionality.
O(Output Format)	[EXACT] Return the test cases in a table with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

Positive Smoke Test Cases – Crackers Shopping Cart Functionality

Test Case ID	Steps	Test Data	Expected Result
TC_SM_001	1. Open the Crackers shopping application. 2. Browse the available crackers. 3. Select a product. 4. Click **Add to Cart**. 5. Open the cart.	Product: Flower Pots; Price: ₹200	The product is added successfully, and the cart displays the correct product name, price, quantity, and total amount.
TC_SM_002	1. Add a cracker product to the cart. 2. Open the cart page.	Product: Sparklers	The cart displays the selected product with the correct details.
TC_SM_003	1. Add a cracker product to the cart. 2. Increase its quantity. 3. Verify the cart total.	Unit price: ₹200; Quantity: 2	The quantity updates to 2, and the subtotal is ₹400.
TC_SM_004	1. Add two different cracker products to the cart. 2. Verify the cart contents and subtotal.	Flower Pots: ₹200; Sparklers: ₹100	Both products appear in the cart, and the subtotal is ₹300.
TC_SM_005	1. Add a cracker product to the cart. 2. Remove the product. 3. Verify the cart.	Product: Flower Pots	The product is removed successfully, and the cart total is recalculated.
TC_SM_006	1. Add available cracker products to the cart. 2. Verify the total amount. 3. Click **Proceed to Checkout**.	Flower Pots: ₹200; Sparklers: ₹100	The cart displays the correct subtotal, and the user proceeds to checkout successfully when all required conditions are met.

- **Scenario 4 — Role-Based Access Control**

Weak Prompt: " Write test cases for user roles."



ICEPOT Prompt:

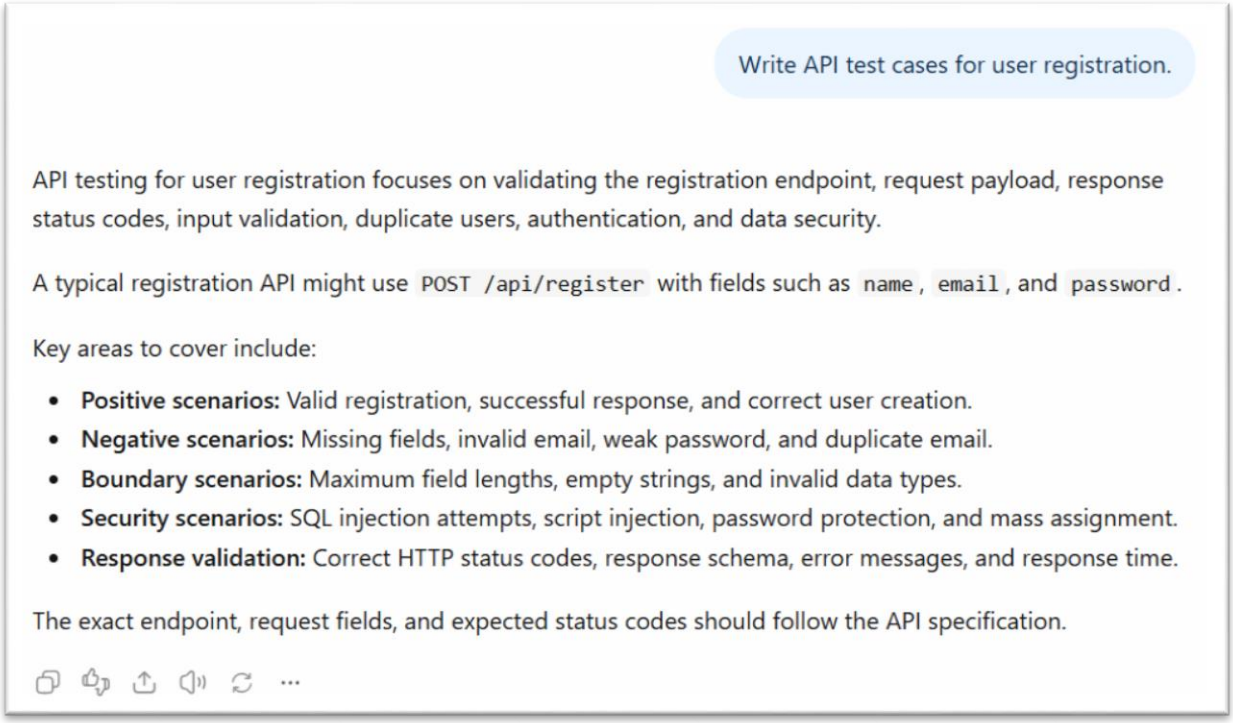
I(Instruction)	[STRICT] [MANDATORY] [NON-DUPLICATE] Generate positive & negative test cases for the Healthcare systems.
C(Context)	[IMPORTANT] The healthcare system supports multiple roles, including patients, doctors, nurses, receptionists, pharmacists, laboratory technicians, billing staff, and administrators. Patients can register, book appointments, view their medical records, and access prescriptions. Doctors can manage appointments, review patient histories, and update diagnoses and treatment plans. Nurses can record vital signs and assist with patient care. Receptionists manage patient registration and appointment scheduling. Pharmacists manage prescriptions and medication dispensing, while laboratory technicians manage test requests and results. Billing staff handle invoices and payments, and administrators manage user accounts, roles, and system configurations. The system must enforce role-based access controls, protect sensitive patient information, and ensure data accuracy and security.
E(Example)	One test case should verify the healthcare system supports multiple roles, including patients, doctors, nurses, receptionists, pharmacists, and administrators. Each role has specific permissions, such as booking appointments, managing patient records, handling prescriptions, and managing user accounts. The system must ensure secure access to patient information and restrict unauthorized actions.
P(Persona)	Act as an experienced software test professional.
O(Output Format)	[EXACT] Return the test cases in a table with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

Test Case ID	Steps	Test Data	Expected Result
TC_001 [POSITIVE]	Register a new patient with valid details.	Name, valid email, phone number	Patient registration is successful.
TC_002 [POSITIVE]	Log in using valid patient credentials.	Valid username and password	Patient logs in and accesses authorized features.
TC_003 [POSITIVE]	Log in as a patient and book an available appointment.	Valid doctor, available date and time	Appointment is booked successfully.
TC_004 [POSITIVE]	Log in as a doctor and view an assigned patient's medical history.	Authorized doctor account; assigned patient	The doctor can view the permitted medical records.
TC_005 [POSITIVE]	Log in as a nurse and record patient vital signs.	Valid patient ID and vital signs	Vital signs are saved successfully.
TC_006 [POSITIVE]	Log in as a receptionist and register a new patient.	Valid patient details	Patient details are saved successfully.
TC_007 [POSITIVE]	Log in as a pharmacist and process a valid prescription.	Valid prescription	The prescription is processed according to dispensing rules.
TC_008 [POSITIVE]	Log in as a laboratory technician and enter test results.	Valid test request and results	Results are saved against the correct patient and test request.
TC_009 [POSITIVE]	Log in as billing staff and generate an invoice.	Patient ID and valid service charges	The invoice is generated with accurate charges.
TC_010 [POSITIVE]	Log in as an administrator and create a user account with an assigned role.	Valid user details and role	The account is created with the correct permissions.

TC_011 [NEGATIVE]	Attempt to register a patient with missing mandatory fields.	Missing name or contact details	Registration is rejected with appropriate validation messages.
TC_012 [NEGATIVE]	Attempt to log in with invalid credentials.	Incorrect username or password	Access is denied, and an appropriate error message is displayed.
TC_013 [NEGATIVE]	Log in as a patient and attempt to access another patient's medical records.	Patient account; another patient's record	Access is denied, and confidential information is not disclosed.
TC_014 [NEGATIVE]	Log in as a receptionist and attempt to modify a doctor's diagnosis.	Receptionist account	The system denies the unauthorized action.
TC_015 [NEGATIVE]	Attempt to book an appointment in an unavailable time slot.	Fully booked date and time	The system rejects the booking and displays an appropriate message.
TC_016 [NEGATIVE]	Attempt to process an invalid or expired prescription.	Invalid or expired prescription	The system prevents dispensing according to prescription validation rules.
TC_017 [NEGATIVE]	Attempt to save laboratory results against a nonexistent patient ID.	Invalid patient ID	The system rejects the entry and displays a validation error.
TC_018 [NEGATIVE]	Log in as billing staff and attempt to change user roles.	Billing staff account	Access is denied because the operation requires appropriate administrative permission.
TC_019 [NEGATIVE]	Log out and attempt to reopen a protected medical record page.	Logged-out session	The system redirects to login or denies access.
TC_020 [NEGATIVE]	Attempt to assign administrator privileges without authorization.	Non-administrator account	The system rejects the request and prevents unauthorized privilege escalation.

- **Scenario 5 — API Testing**

Weak Prompt: "Write API test cases for user registration."



ICEPOT Prompt:

I(Instruction)	[STRICT] [MANDATORY] [NON-DUPLICATE] [POSITIVE] [NEGATIVE] [EDGE] Generate positive, negative, and edge API test cases for user registration.
C(Context)	[IMPORTANT] The e-commerce application allows new users to register by submitting details such as name, email address, password, and phone number through a registration API. The API validates mandatory fields, email and phone number formats, password strength, and duplicate accounts. It creates a user account when valid details are provided and returns appropriate HTTP status codes and response messages. The system must securely store passwords, reject invalid requests, prevent duplicate registrations, and handle boundary conditions and malicious inputs. The registration API should return appropriate HTTP status codes based on the request outcome. Use 201 Created for successful registration, 400 Bad Request or 422 Unprocessable Content for invalid input, 409 Conflict for duplicate email registration, 401 Unauthorized for unauthenticated requests when authentication is required, 405 Method Not Allowed for unsupported HTTP methods, and 500 Internal Server Error for unexpected server-side failures. The exact codes must follow the API specification.

E(Example)	Verify that a user can successfully register in the e-commerce application by submitting valid name, email, password, and phone number through the registration API, and receives a success response with the appropriate HTTP status code.
P(Persona)	Act as a test professional with good domain knowledge.
O(Output Format)	[EXACT] Return the test cases in a table with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

User Registration API Test Cases

Test Case ID	Steps	Test Data & Expected Result
TC_REG_001 [POSITIVE]	Send a POST request with valid registration details.	**Test Data**: Name: Ravi Kumar, Email: ravi.test@example.com, Password: Test@12345, Phone: 9876543210. **Expected Result:** Account is created; HTTP **201 Created** is returned with a success message.
TC_REG_002 [POSITIVE]	Register a user with a valid email address using a different domain.	**Test Data**: Email: user@gmail.com; other fields valid. **Expected Result:** Registration succeeds with HTTP **201 Created**.
TC_REG_003 [POSITIVE]	Register a user with valid details at the minimum permitted field lengths.	**Test Data**: Values meeting all configured minimum-length requirements. **Expected Result:** Registration succeeds if all validation rules are satisfied; HTTP **201 Created**.
TC_REG_004 [POSITIVE]	Register a user with valid details at the maximum permitted field lengths.	**Test Data**: Name and other fields at their configured maximum lengths. **Expected Result:** Registration succeeds if all limits and validation rules are satisfied; HTTP **201 Created**.
TC_REG_005 [NEGATIVE]	Submit a registration request without the name field.	**Test Data**: Name omitted; other fields valid. **Expected Result**: Registration is rejected; HTTP **400 Bad Request** or **422 Unprocessable Content** with a validation message.
TC_REG_006 [NEGATIVE]	Submit a registration request without the email field.	**Test Data**: Email omitted; other fields valid. **Expected Result**: Registration is rejected with the specified validation status and message.

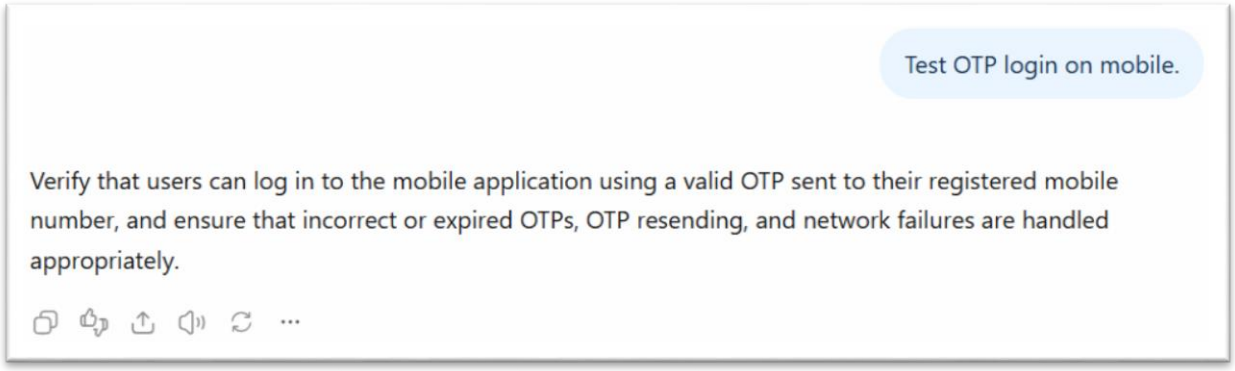
TC_REG_007 [NEGATIVE]	Submit a registration request without the password field.	**Test Data:** Password omitted; other fields valid. **Expected Result:** Registration is rejected with HTTP **400** or **422**.
TC_REG_008 [NEGATIVE]	Submit a registration request with an invalid email format.	**Test Data:** Email: `ravi@invalid` ; other fields valid. **Expected Result:** Registration is rejected with HTTP **400** or **422** and an email validation message.
TC_REG_009 [NEGATIVE]	Submit a registration request with a weak password.	**Test Data:** Password: `123` ; other fields valid. **Expected Result:** Registration is rejected with HTTP **400** or **422** and a password validation message.
TC_REG_010 [NEGATIVE]	Register using an email address that already exists.	**Test Data:** Email: `existing@example.com` . **Expected Result:** Duplicate registration is rejected with HTTP **409 Conflict**; no duplicate account is created.
TC_REG_011 [NEGATIVE]	Submit a registration request with an invalid phone number.	**Test Data:** Phone: `12AB45` ; other fields valid. **Expected Result:** Registration is rejected with HTTP **400** or **422** and a phone validation message.
TC_REG_012 [NEGATIVE]	Submit a request with empty strings in mandatory fields.	**Test Data:** Name: `""` , Email: `""` , Password: `""` . **Expected Result:** Request is rejected with HTTP **400** or **422**; no account is created.
TC_REG_013 [NEGATIVE]	Submit a registration request with an invalid JSON body.	**Test Data:** Malformed JSON payload. **Expected Result:** Request is rejected with HTTP **400 Bad Request**; no account is created.
TC_REG_014 [NEGATIVE]	Submit a registration request using an unsupported HTTP method.	**Test Data:** Use GET instead of POST. **Expected Result:** HTTP **405 Method Not Allowed** is returned if GET is unsupported.

TC_REG_015 [NEGATIVE]	Submit a request without required authentication when the API requires it.	**Test Data:** Valid registration payload without required credentials. **Expected Result:** HTTP **401 Unauthorized** is returned if authentication is required by the API specification.
TC_REG_016 [EDGE]	Submit a name exceeding the maximum permitted length by one character.	**Test Data:** Name length = configured maximum + 1. **Expected Result:** Request is rejected with HTTP **400** or **422**; no account is created.
TC_REG_017 [EDGE]	Submit phone numbers at the minimum and maximum permitted lengths.	**Test Data:** Phone lengths at configured boundaries. **Expected Result:** Boundary values are accepted or rejected according to the defined validation rules.
TC_REG_018 [EDGE]	Submit two simultaneous registration requests using the same email address.	**Test Data:** Two concurrent POST requests with the same email. **Expected Result:** Only one account is created; the other request receives HTTP **409 Conflict** or the documented duplicate response.
TC_REG_019 [EDGE]	Submit a registration request with a numeric value instead of a string for the name.	**Test Data:** Name: `12345`. **Expected Result:** Request is rejected with HTTP **400** or **422** if the schema requires a string.
TC_REG_020 [EDGE]	Submit a name containing a SQL injection-style input.	**Test Data:** Name: `' OR '1'='1`. **Expected Result:** Input is safely handled or rejected; no unintended database operations occur.
TC_REG_021 [EDGE]	Submit a registration request containing unexpected additional fields.	**Test Data:** Valid fields plus `"isAdmin": true`. **Expected Result:** The API rejects or ignores unsupported fields according to the schema; users cannot grant themselves elevated privileges.
TC_REG_022 [EDGE]	Inspect the registration response for sensitive information.	**Test Data:** Submit valid registration details. **Expected Result:** The response does not expose the password, password hash, or internal sensitive data.

TC_REG_023 [EDGE]	Verify secure password storage after successful registration.	**Test Data:** Password: `Test@12345`. **Expected Result:** The password is stored using an approved secure password-hashing method and is never stored as plaintext.
TC_REG_024 [EDGE]	Simulate an unexpected server-side failure during registration.	**Test Data:** Trigger a controlled server error in a test environment. **Expected Result:** HTTP **500 Internal Server Error** is returned with a safe error message; sensitive implementation details are not exposed.

- **Scenario 6 — Mobile App (OTP Login)**

Weak Prompt: "Test OTP login on mobile."



ICEPOT Prompt:

I(Instruction)	[STRICT] [MANDATORY] [NON-DUPLICATE] Generate regression test cases covering the happy flow of OTP login functionality in a mobile application.
C(Context)	[IMPORTANT] The mobile application allows registered users to log in using a one-time password (OTP) sent to their registered mobile number. The system validates the OTP, authenticates users upon successful verification, and navigates them to the home screen. The OTP must be delivered successfully, remain valid for the specified duration, and support the expected login flow across supported mobile devices.
E(Example)	The mobile app allows users to log in using an OTP sent to their registered mobile number. Users can enter the OTP and log in successfully to access the home screen.
P(Persona)	Act as a QA Lead with strong domain knowledge and a clear understanding of business requirements and application functionality.
O(Output Format)	[EXACT] Return the test cases in a table with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

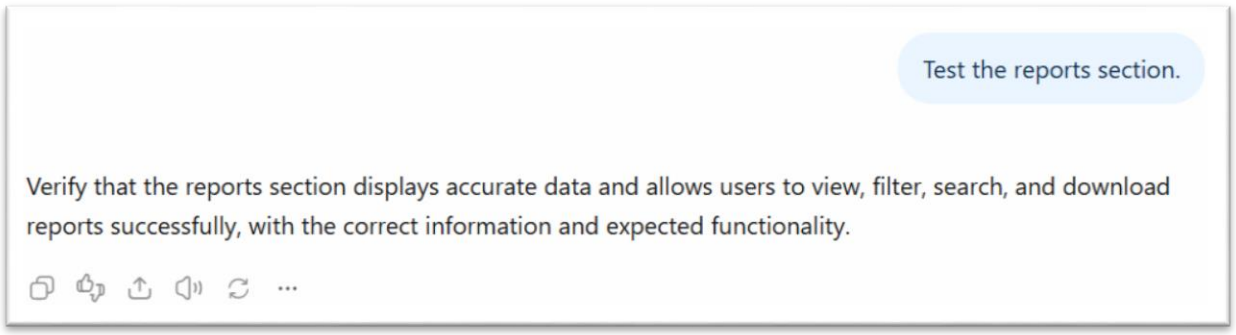
Regression Happy Flow Test Cases – OTP Login (Mobile App)

Test Case ID	Steps	Test Data & Expected Result
TC_OTP_001	1. Open the mobile app. 2. Enter a registered mobile number. 3. Tap **Get OTP**. 4. Enter the received OTP. 5. Tap **Login**.	**Test Data**: Registered mobile number and valid OTP. **Expected Result**: OTP is verified, login succeeds, and the user navigates to the home screen.
TC_OTP_002	1. Open the login screen. 2. Enter a registered mobile number. 3. Request an OTP.	**Test Data**: Valid registered mobile number. **Expected Result**: OTP is sent successfully, and the OTP entry screen is displayed.
TC_OTP_003	1. Request an OTP. 2. Retrieve the OTP from the received message. 3. Enter it in the OTP field.	**Test Data**: Valid OTP. **Expected Result**: OTP digits are accepted correctly in the input field.
TC_OTP_004	1. Request an OTP. 2. Enter the valid OTP before it expires. 3. Submit the OTP.	**Test Data**: Valid, unexpired OTP. **Expected Result**: OTP verification succeeds, and the user is authenticated.
TC_OTP_005	1. Complete OTP login successfully. 2. Verify the landing screen.	**Test Data**: Registered user credentials and valid OTP. **Expected Result**: The home screen loads successfully with the correct user session.
TC_OTP_006	1. Request an OTP. 2. Enter the valid OTP using the mobile keyboard. 3. Submit the OTP.	**Test Data**: Valid OTP entered through the device keyboard. **Expected Result**: OTP is entered and submitted correctly without UI issues.
TC_OTP_007	1. Request an OTP. 2. Allow the device to receive the OTP message. 3. Use OTP auto-fill if supported. 4. Submit the OTP.	**Test Data**: Valid OTP received through SMS. **Expected Result**: OTP auto-fill works as expected, and login succeeds.
TC_OTP_008	1. Log in using a valid OTP. 2. Navigate betwe…he home screen and another screen.	**Test Data**: Successfully authenticated user session. **Expected Result**: The session remains active, and navigation works correctly.
TC_OTP_009	1. Complete OTP login on a supported mobile device. 2. Repeat the flow on another supported device or OS version.	**Test Data**: Registered mobile number and valid OTP. **Expected Result**: OTP login works consistently across supported devices and operating systems.
TC_OTP_010	1. Log in successfully using OTP. 2. Close and reopen the app according to the expected session behavior.	**Test Data**: Authenticated user session. **Expected Result**: The app handles the existing session according to the specified session-management requirements.

Note: These test cases cover successful OTP login flows and key regression checks. OTP expiry, invalid OTPs, resend failures, and other negative scenarios are excluded because the request specifically focuses on the happy flow.

- **Scenario 7 — Dashboard / Report Generation**

Weak Prompt: "Test the reports section."



ICEPOT Prompt:

I(Instruction)	[STRICT] [MANDATORY] [NON-DUPLICATE] Generate functional and non-functional test cases for the reports section of School management application.
C(Context)	[IMPORTANT] The school management application provides a Reports section where users can view, filter, generate, and download reports related to student attendance, academic performance, fees, and staff details. The system should display accurate data, respond within the expected time, protect sensitive information, and work correctly across supported devices and browsers.
E(Example)	The school management application allows users to view student attendance reports, check academic performance, generate fee reports, and download reports in PDF or Excel format. The reports should display accurate information and load efficiently.
P(Persona)	Act as an Associate Professional with strong domain knowledge and a clear understanding of business requirements and application functionality.
O(Output Format)	[EXACT] Return the test cases in an excel format with Test Case IDs, Steps, Test Data & Expected Result.
T(Tone)	[PROFESSIONAL] [CONCISE] Stick with technical standards.

Analyzed

The Excel file is ready with **16 functional and non-functional test cases** for the School Management application's Reports section.

Download the Excel file (.xlsx) ↗

The spreadsheet contains exactly these four columns:

- Test Case ID
- Steps
- Test Data
- Expected Result

The test cases cover attendance, academic performance, fee reports, filtering, searching, PDF/Excel exports, performance, security, compatibility, and accessibility.

School_Management_Reports_Test_Cases.xlsx
Excel

ChatGPT can make mistakes. Check important info.

Ask ChatGPT Think

	A	B	C
1	**Test Case ID**	**Steps**	**Test Data**
2	TC_REP_F_001	1. Open Reports. 2. Select Student Attendance Report. 3. Click Generate Report.	Current month's attendance re
3	TC_REP_F_002	1. Open Reports. 2. Select Academic Performance Report. 3. Choose a class and term. 4. Generate the report.	Class 10; Term 1 results.
4	TC_REP_F_003	1. Open Reports. 2. Select Fee Report. 3. Choose a date range. 4. Generate the report.	Fee records for the current mo
5	TC_REP_F_004	1. Open a report. 2. Select a start date and end date. 3. Apply the filter.	01-10-2026 to 15-10-2026.
6	TC_REP_F_005	1. Open the relevant report. 2. Enter a student name or ID in the search field. 3. Run the search.	Student ID: STU1001.
7	TC_REP_F_006	1. Generate a report. 2. Select Download PDF. 3. Open the downloaded file.	Student Attendance Report.
8	TC_REP_F_007	1. Generate a Fee Report. 2. Select Export to Excel. 3. Open the downloaded spreadsheet.	Class 10 fee records.